

Student name: Nguyen Khanh Ly - 11247193

Class: DS66B

Introduction to Query Optimization

Lab Exercise 1: Triggering Nested Loop Join

The screenshot shows a database query optimizer interface. At the top, there's a 'Query' tab with a query history icon. The query text is as follows:

```
13 EXPLAIN (ANALYZE, BUFFERS)
14 SELECT c.first_name, c.last_name, r.rental_date
15 FROM customer c
16 JOIN rental r ON c.customer_id = r.customer_id
17 WHERE c.customer_id = 1;
```

Below the query is a 'Data Output' tab with a 'Messages' and 'Notifications' icon. The interface includes a toolbar with icons for saving, undo, redo, and other functions. The 'Showing rows: 1 to 14' and 'Page No: 1 of 1' are displayed. The main area shows the 'QUERY PLAN' text:

```
1 Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.794..3.546 rows=37.00 loops=1)
2   Buffers: shared hit=1 read=152
3   -> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.032..0.034 rows=1.00 loops=1)
4     Index Cond: (customer_id = 1)
5     Index Searches: 1
6     Buffers: shared hit=1 read=2
7   -> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.759..3.501 rows=37.00 loops=1)
8     Filter: (customer_id = 1)
9     Rows Removed by Filter: 16012
10    Buffers: shared read=150
11  Planning:
12    Buffers: shared hit=85 read=5
13  Planning Time: 8.597 ms
14  Execution Time: 3.575 ms
```

At the bottom, it shows 'Total rows: 14' and 'Query complete 00:00:00.071'. The status bar indicates 'CRLF' and 'Ln 16, Col 43'.

Analysis Questions:

- Identify the Roles: Look at the plan tree. Which table is the Outer (top) and which is the Inner (bottom)?**
 - Outer (top): Index Scan using customer_pkey on customer c (returning 1 row)
 - Inner (bottom): Seq Scan on rental r (returning 37 rows)
- Why Nested Loop? How many rows did the customer table return? Why does this make Hash Join "too expensive"?**
 - The customer table returns only **1 row** due to the equality filter (customer_id = 1).
 - A Nested Loop is chosen because scanning for a single matching row is extremely cheap, whereas building a hash table for a hash join would introduce unnecessary overhead and complexity.
- Efficiency: Look at loops in the Inner Table node. Why is it 1?**
 - The loop value for the Inner Table (rental) is 1 because the Outer Table (customer) only feeds **a single row** into the loop, requiring the inner scan to execute only once.

Lab Exercise 2: Triggering Hash Join

The screenshot shows a database query interface with a query editor at the top and a query plan viewer below. The query editor contains the following SQL code:

```
19 -- tab z
20 EXPLAIN (ANALYZE, BUFFERS)
21 SELECT f.title, l.name AS language
22 FROM film f
23 JOIN language l ON f.language_id = l.language_id;
```

The query plan viewer displays the following details:

- QUERY PLAN**
- 1 Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.932..1.282 rows=1000.00 loops=1)
- 2 Hash Cond: (f.language_id = l.language_id)
- 3 Buffers: shared hit=88 read=1
- 4 -> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.017..0.223 rows=1000.00 loops=1)
- 5 Buffers: shared hit=88
- 6 -> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.904..0.904 rows=6.00 loops=1)
- 7 Buckets: 1024 Batches: 1 Memory Usage: 9kB
- 8 Buffers: shared read=1
- 9 -> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.894..0.896 rows=6.00 loops=1)
- 10 Buffers: shared read=1
- 11 Planning:
- 12 Buffers: shared hit=166 read=1
- 13 Planning Time: 5.404 ms
- 14 Execution Time: 1.335 ms

At the bottom of the interface, it shows "Total rows: 14" and "Query complete 00:00:00.067". The status bar indicates "CRLF" and "Ln 23, Col 50".

Analysis Questions:

- Identify Phases: Which table is being Hashed (Built)? Which table is being Probed?**
 - **Hashed (Built):** Seq Scan on language l (by the Hash node)
 - **Probed:** Seq Scan on film f
- Memory Check: Look at the Hash node. What is the Memory Usage?**
 - Memory Usage: **9kB** (with Buckets: 1024 & Batches: 1)
- Hypothesis: What happens if you run SET work_mem = '64kB'; and run it again? (Try it!)**

Since **9kB** (current) is already well **below 64kB**, running it again with 64kB will still fit entirely in memory (Batches: 1), resulting in **no change** in execution strategy or performance.

→ To force a spill to disk, work_mem would need to be set lower than 9kB

Query
Query History

```

25
26 SET work_mem = '64kB';
27
28 EXPLAIN (ANALYZE, BUFFERS)
29 SELECT f.title, l.name AS language
30 FROM film f
31 JOIN language l ON f.language_id = l.language_id;

```

Data Output
Messages
Notifications

Showing rows: 1 to 12
Page No: 1 of 1

QUERY PLAN	
text	
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.062..0.489 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.029..0.220 rows=1000.00 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.016..0.017 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.009..0.011 rows=6.00 loops=1)
10	Buffers: shared hit=1
11	Planning Time: 0.249 ms
12	Execution Time: 0.570 ms

Total rows: 12
Query complete 00:00:00.049
CRLF
Ln 31, Col 50

Lab Exercise 3: Triggering Merge Join

```

-- Disable other join types to force Merge Join
SET enable_hashjoin = OFF;
SET enable_nestloop = OFF;

EXPLAIN (ANALYZE, BUFFERS)
SELECT f.film_id, f.title, i.inventory_id
FROM film f
JOIN inventory i ON f.film_id = i.film_id
ORDER BY f.film_id; -- Explicit sort requirement

```

dvdrental/postgres@PostgreSQL 18	
No limit	
Data Output Messages Notifications	
Showing rows: 1 to 19 Page No: 1 of 1	
1	QUERY PLAN
2	text
3	1 Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=1.514..3.021 rows=4581.00 loops=1)
4	2 Merge Cond: (f.film_id = i.film_id)
5	3 Buffers: shared hit=88 read=25, temp read=10 written=12
6	4 -> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.004..0.378 rows=1000.00 loops=1)
7	5 Index Searches: 1
8	6 Buffers: shared hit=84 read=1
9	7 -> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=1.505..2.095 rows=4581.00 loops=1)
10	8 Storage: Memory Maximum Storage: 17kB
11	9 Buffers: shared hit=4 read=24, temp read=10 written=12
12	10 -> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=1.502..1.760 rows=4581.00 loops=1)
13	11 Sort Key: i.film_id
14	12 Sort Method: external merge Disk: 80kB
15	13 Buffers: shared hit=4 read=24, temp read=10 written=12
16	14 -> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.025..0.458 rows=4581.00 loops=1)
17	15 Buffers: shared hit=1 read=24
18	16 Planning:
19	17 Buffers: shared hit=43 read=1
	18 Planning Time: 2.006 ms
	19 Execution Time: 8.473 ms
Total rows: 19 Query complete 00:00:00.044	
CRLF Ln 37, Col 1	

Analysis Questions:

- Structure: Do you see a Sort node? Or did it use Index Scan to get sorted data for free?**
 - Yes, I can see an explicit Sort node (along with a Materialize node) for the inventory table (Sort Key: i.film_id, using an external merge disk of 80kB).
 - Meanwhile, the film table utilized an Index Scan (using film_pkey), which provided its side of the data already sorted for free.
- Compare: Reset the settings (RESET ALL;) and remove the ORDER BY. Does it switch back to Hash Join? Why?**
 - Yes, it switches back to a Hash Join.
 - Why: Without the forced settings (enable_hashjoin and enable_nestloop) and without an explicit ORDER BY requirement, PostgreSQL's cost-based optimizer determines that building an in-memory hash table is cheaper and more efficient than incurring the CPU and disk I/O costs required to sort the datasets for a Merge Join.